Object Oriented Design Assignment

MD. NAZMUL HASAN

Department of Computer Science and Engineering

University of Rajshahi

Student ID: 2111176131

Question 1

Explain the benefit of representing real-world entities—such as Citizen, MunicipalService, and Complaint—in the Dhaka Urban Management System as objects in your code.

Answer

Representing real-world entities like Citizen, MunicipalService, and Complaint as objects in the code provides several important benefits in object-oriented design:

- **Natural Mapping to the Problem Domain**: The real world has citizens who file complaints, municipal services that handle those complaints, and complaint records themselves. By creating classes like Citizen, MunicipalService, and Complaint, the code structure directly mirrors the real-world concepts. This makes the system easier to understand for developers, stakeholders, and future maintainers.
- **Encapsulation**: Each object can encapsulate its own data (attributes) and behavior (methods). For example, a Citizen object can store personal details (ID, name, address) and provide methods like login() or submitComplaint(). Sensitive data is hidden and only accessible through controlled interfaces, improving security and maintainability.
- **Reusability**: Once defined, these objects can be reused across different parts of the system. A Complaint object created for waste collection can be reused for road maintenance requests with minimal changes.
- **Modularity and Maintainability**: Changes to one entity affect only its class. If the complaint submission process changes, we only modify the Complaint class or related methods without touching unrelated parts of the code.
- **Clear Relationships and Interactions**: Objects can collaborate naturally—e.g., a Citizen object creates and sends a Complaint object to a MunicipalService object. This models real-world interactions accurately using associations, aggregation, or composition.

Overall, this approach leads to cleaner, more intuitive, and scalable code that closely reflects the actual urban management processes.

Question 2

Explain the benefit of representing real-world entities—such as Farmer, SensorModule, and IrrigationController—in the Automated Agricultural System as objects in your code.

Answer

Modeling real-world entities such as Farmer, SensorModule, and IrrigationController as objects brings the following key benefits:

- **Direct Correspondence to Reality**: In agriculture, a farmer monitors and controls equipment, sensors collect environmental data, and controllers operate irrigation pumps. Creating classes Farmer, SensorModule, and IrrigationController makes the code structure reflect the physical and operational reality of the farm, making it intuitive for developers and domain experts.
- **Encapsulation of State and Behavior**: Each object bundles its data and operations. A SensorModule can hold readings (soil moisture, temperature) and provide methods like readMoisture() or sendData(). An IrrigationController can manage pump state and expose startIrrigation() or stopIrrigation(). This hides implementation details and prevents direct manipulation of internal state.
- **Reusability and Extensibility**: Sensor modules of different types can inherit from a base SensorModule class. New controllers or sensors can be added without rewriting existing code.
- **Modifiability**: If irrigation logic changes (e.g., new scheduling algorithm), only the IrrigationController class needs updating, leaving Farmer and SensorModule untouched.
- **Natural Collaboration**: Objects interact as they do in the real world—a Farmer queries a SensorModule for data and commands an IrrigationController based on that data. This collaboration is expressed clearly through method calls and associations.

This object-oriented modeling results in a flexible, maintainable, and realistic representation of the automated agricultural system.

Question 3

Consider a Dhaka Urban Management System scenario: a city resident logs into the smart city portal using their ID and password to verify their identity with the municipal database. The user is then provided options to report waste collection issues, pay utility bills, request road maintenance, or track service status. Explain how CRC (Class–Responsibility–Collaborator) cards can be utilized to isolate and define the system components, including their attributes, functionalities, and interactions — for example, classes such as Citizen, MunicipalService, Complaint, and Payment can be defined along with their responsibilities and collaborating relationships.

Answer

CRC (Class–Responsibility–Collaborator) cards are an effective technique in the early stages of object-oriented design to identify classes, assign clear responsibilities, and define how they collaborate. Each card represents one class and lists its responsibilities (what it knows or does) and collaborators (other classes it interacts with).

Here is how CRC cards can be applied to the Dhaka Urban Management System:

Example CRC Cards:

1. **Class: Citizen**
    - **Responsibilities**:
        - Know personal information (ID, name, address, password)
        - Authenticate/login to the portal
        - Submit a complaint
        - Make a payment
        - Track complaint/payment status
    - **Collaborators**: AuthenticationService, Complaint, Payment, MunicipalService
2. **Class: Complaint**
    - **Responsibilities**:
        - Know complaint details (type, description, location, status, timestamp)
        - Validate complaint data
        - Update status (e.g., submitted → in-progress → resolved)
        - Notify citizen of status changes
    - **Collaborators**: Citizen, MunicipalService
3. **Class: Payment**
    - **Responsibilities**:
        - Know payment details (amount, bill type, date, transaction ID)
        - Process payment
        - Generate receipt
        - Update bill status to paid

- **Collaborators**: Citizen, PaymentGateway (external), MunicipalService

4. **Class: MunicipalService**
    - **Responsibilities**:
        - Manage different service categories (waste, roads, utilities)
        - Receive and route complaints
        - Assign complaints to departments
        - Provide status updates
        - Maintain bill records
    - **Collaborators**: Complaint, Payment, Citizen
5. **Class: AuthenticationService** (optional supporting class)
    - **Responsibilities**:
        - Verify citizen ID and password against municipal database
        - Issue session/token upon successful login
    - **Collaborators**: Citizen

How CRC cards help isolate and define components:

- **Isolation**: Each card forces us to focus on a single class and clearly separate its concerns from others.
- **Responsibility Assignment**: By listing only what the class "knows" and "does," we avoid overloading classes and follow the Single Responsibility Principle.
- **Interaction Discovery**: The collaborator column explicitly shows dependencies. For example, when a Citizen submits a complaint, we see that Citizen collaborates with Complaint, which in turn collaborates with MunicipalService.
- **Iterative Refinement**: During brainstorming sessions, team members can role-play scenarios (e.g., "Citizen wants to report waste issue") and walk through which cards activate and how they pass messages, revealing missing classes or misplaced responsibilities.

By systematically creating and reviewing CRC cards, we arrive at a well-structured design where classes have clear roles and well-defined interactions, forming a solid foundation for subsequent UML diagrams and implementation.

Question 4

What is the difference between software design and software architecture?

Answer

Aspect	Software Architecture	Software Design
Scope	High-level structure of the entire system; focuses on major components and their interactions	Detailed internal structure of individual components or modules
Level of Abstraction	Abstract; concerned with system-wide qualities (scalability, performance, security)	More concrete; focuses on classes, methods, algorithms, data structures
Concerns	Non-functional requirements, technology choices, architectural styles (e.g., microservices, layered)	Functional requirements, detailed behavior, interfaces within components
Primary Artifacts	Architecture diagrams, component diagrams, deployment views, architectural patterns	Class diagrams, sequence diagrams, detailed design documents, pseudocode
Decision Impact	Long-term, hard to change (affects overall system cost and quality attributes)	Shorter-term, easier to refactor within the architectural boundaries
Stakeholders	Architects, senior developers, project managers, sometimes clients	Developers, designers, testers
Timing	Early in the project (during requirements and high-level planning)	Follows architecture; performed during detailed design phase

In summary, **software architecture** defines the big picture and foundational structure of the system, while **software design** fills in the detailed implementation within that structure. Good architecture enables flexible and effective design.

Question 5

Consider an Automated Agricultural System scenario: a farmer accesses the automated control system through a mobile app, verifies their identity using a registered farm ID, and is then given options to monitor soil moisture, control irrigation pumps, check crop growth data, or analyze weather forecasts. Explain how CRC (Class–Responsibility–Collaborator) cards can be used to isolate and define the system components, including their attributes, functionalities, and interactions — for instance, classes like Farmer, SensorModule, IrrigationController, and

WeatherService can be defined to represent various responsibilities and their collaborations in the automated process.

Answer

CRC cards help systematically identify classes, assign focused responsibilities, and uncover collaborations in the Automated Agricultural System.

Example CRC Cards:

1. **Class: Farmer**
    - **Responsibilities**:
        - Know personal/farm details (farm ID, name, registered fields)
        - Authenticate via mobile app
        - View sensor data
        - Issue irrigation commands
        - View weather forecasts and crop data
    - **Collaborators**: AuthenticationService, SensorModule, IrrigationController, WeatherService, CropDataAnalyzer
2. **Class: SensorModule**
    - **Responsibilities**:
        - Know sensor readings (soil moisture, temperature, humidity, etc.)
        - Collect and store real-time data
        - Provide current/latest readings on request
        - Detect threshold breaches and send alerts
    - **Collaborators**: Farmer, IrrigationController (optional for automatic mode)
3. **Class: IrrigationController**
    - **Responsibilities**:
        - Know pump/actuator status
        - Start/stop irrigation based on commands or sensor data
        - Schedule irrigation if configured
        - Log irrigation events

- **Collaborators**: Farmer, SensorModule

4. **Class: WeatherService**
    - **Responsibilities**:
        - Fetch current and forecast weather data (from external API)
        - Provide formatted weather information
        - Issue alerts for adverse conditions
    - **Collaborators**: Farmer
5. **Class: CropDataAnalyzer** (optional)
    - **Responsibilities**:
        - Store historical crop growth data
        - Generate growth reports and predictions
        - Combine sensor and weather data for insights
    - **Collaborators**: Farmer, SensorModule, WeatherService
6. **Class: AuthenticationService** (supporting)
    - **Responsibilities**:
        - Verify farm ID and credentials
        - Grant session access to the mobile app
    - **Collaborators**: Farmer

Benefits of using CRC cards in this scenario:

- **Clear Separation of Concerns**: Each class has well-defined responsibilities (e.g., SensorModule only handles data collection, not actuation).
- **Explicit Interactions**: Collaborators show how data flows—Farmer requests readings from SensorModule, then decides to command IrrigationController.
- **Scenario Walk-through**: We can simulate user stories (e.g., "Farmer checks soil moisture and starts irrigation") by activating relevant cards in sequence, ensuring all necessary responsibilities and collaborations are covered.
- **Early Detection of Issues**: Missing classes (e.g., a central Dashboard or NotificationService) or overloaded classes become evident during CRC sessions.

Question 6

The design principle of ‘decomposition’ takes a whole thing and divides it into different parts. There are three types of relationships in decomposition, which define the interaction between the whole and the parts: association, aggregation and composition. Explain those relationships and also draw the UML diagram for each of these relationships based on the Dhaka Urban Management System concept.

Answer

In object-oriented design, decomposition breaks down a system into smaller, manageable parts (objects/classes). The three main relationships that define how the "whole" interacts with its "parts" are **association**, **aggregation**, and **composition**. These differ in strength of ownership and lifetime dependency.

1. **Association**
    - **Explanation**: A general relationship where one object "uses" or "knows about" another object. There is no ownership; both objects have independent lifetimes. It can be unidirectional or bidirectional.
    - **Example in Dhaka Urban Management System**: A Citizen submits a Complaint. The citizen can exist without any complaint, and complaints are meaningful only in relation to citizens/services.
    - **UML Notation**: Simple line between classes (arrow for unidirectional).

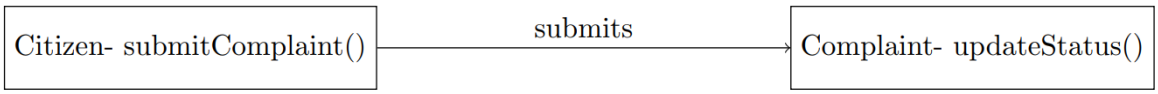


Figure 1: Association: Citizen submits Complaint

2. Aggregation

- **Explanation**: A "whole-part" relationship where the part can exist independently of the whole (weak ownership). If the whole is destroyed, the parts can still live and be shared with other wholes. Denoted by a hollow diamond on the whole side.
- **Example**: A MunicipalService manages multiple Complaint objects (e.g., waste collection department handles many complaints). Complaints can be reassigned to another service if needed.
- **UML Notation**: Hollow diamond on the whole.

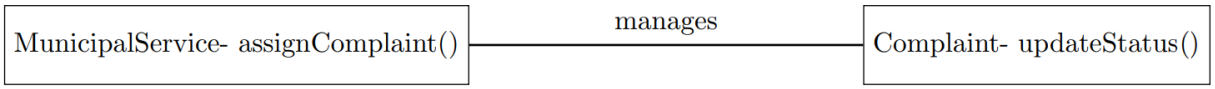


Figure 2: Aggregation: MunicipalService manages Complaints

2. **Composition**

   - **Explanation**: A stronger "whole-part" relationship where the part cannot exist without the whole (strong ownership). If the whole is destroyed, the parts are also destroyed. Denoted by a filled diamond on the whole side.
   - **Example**: A Complaint contains multiple Attachment objects (photos/documents uploaded by the citizen). If the complaint is deleted, its attachments must also be deleted.
   - **UML Notation**: Filled diamond on the whole.

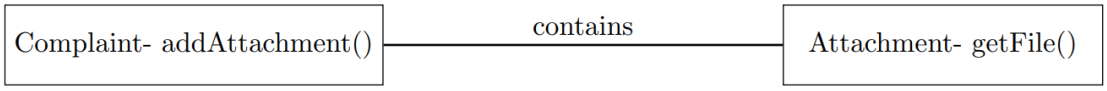


Figure 3: Composition: Complaint contains Attachments

These relationships help model real-world dependencies accurately and guide proper object lifetime management in the Dhaka Urban Management System.

Question 7

The computation steps taken within a class never need to be known by any other class, as long as they are able to access the interface. What type of encapsulation occurs here? Explain with a potential example.

Answer

The type of encapsulation described here is **information hiding** (also known as **implementation hiding** or **data abstraction**).

Explanation: Encapsulation in OOP bundles data and methods together while restricting direct access to internal details. Information hiding goes further by concealing the internal implementation (how things are done) and exposing only the public interface (what can be done). Other classes interact through well-defined methods without knowing the internal computation steps, algorithms, or data structures used.

This promotes loose coupling, easier maintenance, and safer refactoring—internal changes don't affect clients as long as the interface remains stable.

Potential Example (from Dhaka Urban Management System):

Consider the Payment class responsible for processing utility bill payments.

- Public interface: processPayment(amount, billID) → returns success/failure.
- Internal computation (hidden):
    - Validate bill details with database
    - Connect to external payment gateway
    - Handle transaction retry logic
    - Update municipal records
    - Generate receipt

A Citizen object calls payment.processPayment(5000, "BILL123"). The citizen (or any other class) does not need to know the complex steps involving gateway APIs, encryption, or database transactions. Only the method signature and its contract matter. If the payment gateway changes tomorrow, only the internal code of Payment is modified—external classes remain unchanged.

This is classic information hiding: the "how" is hidden behind the "what" provided by the interface.

Question 8

In the case of an Agricultural Automation System, design a parent class FarmingMachine, and child classes IrrigationSystem and Harvester — where the child classes inherit attributes and behaviours from the parent class. Now, explain what type of design principle is used here.

Answer

Designed Class Hierarchy (in conceptual pseudocode form):

Java

```
class FarmingMachine {
   // Common attributes
   protected String machineID;
   protected String manufacturer;
   protected boolean isPoweredOn;
   protected double currentFuelLevel;  // or battery level

   // Common behaviours
   public void start() {
```

```
        if (!isPoweredOn) {
            isPoweredOn = true;
            System.out.println("Machine " + machineID + " started.");
        }
    }

    public void stop() {
        if (isPoweredOn) {
            isPoweredOn = false;
            System.out.println("Machine " + machineID + " stopped.");
        }
    }

    public double checkFuelLevel() {
        return currentFuelLevel;
    }

    public void performMaintenance() {
        System.out.println("Performing routine maintenance on " + machineID);
    }
}

class IrrigationSystem extends FarmingMachine {
    private double waterFlowRate;
    private boolean pumpActive;

    // Specific behaviour
```

```
    public void activatePump(double duration) {
        if (isPoweredOn) {
            pumpActive = true;
            System.out.println("Irrigation pump activated for " + duration + " minutes.");
        }
    }

    public void adjustFlowRate(double rate) {
        waterFlowRate = rate;
        System.out.println("Flow rate set to " + rate + " L/min.");
    }
}

class Harvester extends FarmingMachine {
    private double cuttingWidth;
    private int harvestedAmount;

    // Specific behaviour
    public void beginHarvesting() {
        if (isPoweredOn) {
            System.out.println("Harvesting started with cutting width: " + cuttingWidth + " meters.");
        }
    }

    public void collectCrop(int amount) {
        harvestedAmount += amount;
        System.out.println("Collected " + amount + " kg. Total: " + harvestedAmount + " kg.");
```

```
    }
}
```

Design Principle Used: **Inheritance** (specifically, **generalization/specialization** or the **"is-a" relationship**).

Explanation: The principle applied here is inheritance, which allows code reuse and establishes a hierarchical relationship. IrrigationSystem and Harvester are specialized types of FarmingMachine—both "are" farming machines. Common attributes (like ID, power status, fuel) and behaviors (start, stop, maintenance) are defined once in the parent and automatically inherited by children, avoiding duplication (following DRY principle). Each child adds its own specific attributes and methods while retaining the parent's functionality.

Question 9

Suppose we have a class called ServiceManager in the Dhaka Urban Management System that has two purposes: handling waste collection requests and road maintenance requests.

Here we have a processRequest() method that takes a zero (0) flag if you want to process a waste collection request, and takes the one (1) flag if you want to process a road maintenance request. The class diagram for the ServiceManager and implementation of its processRequest() method is as below:

ServiceManager

- processRequest(controlFlag: int)

Java

```
public void processRequest(int controlFlag) {
   switch (controlFlag) {
      case 0:
         return this.handleWasteCollection();
      case 1:
         return this.handleRoadMaintenance();
      default:
         throw new UnknownControlFlagException();
   }
}
```

a) Evaluate the above design in terms of coupling and cohesion. Give justification in each case. (5 marks)

Answer

- **Cohesion**: **Low (Procedural/Functional cohesion at best, but actually closer to Logical cohesion)** Justification: The ServiceManager class is responsible for two unrelated tasks—waste collection and road maintenance. These are distinct municipal services with different business rules, data, and workflows. The single method processRequest() uses a control flag (an integer) to decide which logically separate function to execute. This is a classic example of **logical cohesion**, where a class groups operations that are logically related in a broad sense (both are "service requests") but not functionally focused on a single, well-defined purpose. Low cohesion makes the class harder to understand, test, and maintain because changes to one service type (e.g., adding new waste rules) may unintentionally affect the other.
- **Coupling**: **Moderate to High (Control coupling)** Justification: The use of a control flag (integer parameter) to direct program flow creates **control coupling**, which is considered one of the worst forms of coupling (apart from content coupling). The caller must know the internal meaning of the flags (0 = waste, 1 = road), tightly binding the client code to the internal decision structure of ServiceManager. If a new service type is added, both the switch statement and all calling code must be modified. This violates the Open/Closed Principle and makes the system fragile.

b) Suggest a better design for the above class that meets acceptable coupling and cohesion requirements. (3.75 marks)

Answer

To achieve **high cohesion** and **low coupling**, we should apply the **Single Responsibility Principle (SRP)** by splitting the responsibilities into separate classes.

Better Design:

Create an interface or abstract class for common behavior (if any), and concrete classes for each specific service:

Java

```
public interface ServiceHandler {
    void processRequest();
}
```

```
public class WasteCollectionHandler implements ServiceHandler {
    public void processRequest() {
        // Specific logic for waste collection
        handleWasteCollection();
    }

    private void handleWasteCollection() { ... }
}

public class RoadMaintenanceHandler implements ServiceHandler {
    public void processRequest() {
        // Specific logic for road maintenance
        handleRoadMaintenance();
    }

    private void handleRoadMaintenance() { ... }
}
```

The ServiceManager can now act as a coordinator (or be eliminated if not needed):

Java

```
public class ServiceManager {
    private Map<String, ServiceHandler> handlers;

    public ServiceManager() {
        handlers = new HashMap<>();
        handlers.put("waste", new WasteCollectionHandler());
        handlers.put("road", new RoadMaintenanceHandler());
    }
```

```
    public void processRequest(String serviceType) {
        ServiceHandler handler = handlers.get(serviceType);
        if (handler != null) {
            handler.processRequest();
        } else {
            throw new UnknownServiceException();
        }
    }
}
```

Benefits:

- **High cohesion**: Each handler class has a single, clear responsibility.
- **Low coupling**: Clients depend on abstractions (ServiceHandler) or meaningful strings, not magic numbers. Adding a new service requires only a new class and registration—no changes to existing code.
- Eliminates control coupling from flags.

Alternatively, if polymorphism is preferred, clients can directly hold references to the specific handler they need, removing the need for a central manager in simple cases.

Question 10

Suppose we have a class called FarmController in the Automated Agricultural System that has two purposes: controlling irrigation and fertilizer dispensing.

Here we have a controlSystem() method that takes a zero (0) flag if you want to activate the irrigation system, and takes the one (1) flag if you want to activate the fertilizer system. The class diagram for the FarmController and implementation of its controlSystem() method is as below:

a) Evaluate the above design in terms of coupling and cohesion. Give justification in each case. (5 marks)

Answer

- **Cohesion**: **Low (Logical cohesion)** Justification: The FarmController class combines two distinct responsibilities: irrigation control and fertilizer dispensing. These are separate agricultural operations with different sensors, actuators, schedules, and safety rules. The

controlSystem() method uses a numeric flag to select which unrelated operation to perform. This groups operations that are conceptually related at a high level ("farm automation") but not focused on a single task. This is logical cohesion, which is weak and leads to a class that is difficult to maintain—changes in irrigation logic risk affecting fertilizer logic unnecessarily.

- **Coupling**: **Moderate to High (Control coupling)** Justification: The integer flag parameter introduces **control coupling**. The calling code must know and use specific magic values (0 or 1) that represent internal control flow decisions inside FarmController. This tightly couples the client to the implementation details. Adding a new subsystem (e.g., pest control) would require modifying both the switch statement and all client code that calls controlSystem(). Such design is brittle and violates extensibility principles.

b) Suggest a better design for the above class that meets acceptable coupling and cohesion requirements.

Answer

Apply the **Single Responsibility Principle** by separating the concerns into dedicated classes.

Improved Design:

Java

```
public interface FarmSubsystem {
   void activate();
   void deactivate();  // optional common behavior
}


public class IrrigationController implements FarmSubsystem {
   public void activate() {
      // Irrigation-specific logic
      startIrrigationPump();
      adjustWaterFlow();
   }

   public void deactivate() {
```

```
        stopIrrigationPump();
    }

    private void startIrrigationPump() { ... }
    // other irrigation methods
}

public class FertilizerDispenser implements FarmSubsystem {
    public void activate() {
        // Fertilizer-specific logic
        openDispenseValve();
        releaseFertilizer();
    }

    public void deactivate() {
        closeDispenseValve();
    }

    private void releaseFertilizer() { ... }
    // other fertilizer methods
}
```

A higher-level FarmController can compose these if coordination is needed:

Java

```
public class FarmController {
    private IrrigationController irrigation;
    private FertilizerDispenser fertilizer;
```

```
    public FarmController() {
        this.irrigation = new IrrigationController();
        this.fertilizer = new FertilizerDispenser();
    }

    public void activateIrrigation() {
        irrigation.activate();
    }

    public void activateFertilizer() {
        fertilizer.activate();
    }
}
```

Benefits:

- **High functional cohesion**: Each class (IrrigationController, FertilizerDispenser) focuses on one clear responsibility.
- **Low coupling**: No control flags; clients call meaningful methods (activateIrrigation()). New subsystems can be added without modifying existing classes.
- Supports polymorphism: A list of FarmSubsystem can be managed uniformly if needed.
- Easier testing, maintenance, and future extension (e.g., adding PestControlSystem).

11. Modern AI systems heavily rely on modularity, abstraction, and scalable architecture. Critically analyze how principles of Object-Oriented Analysis and Design (OOAD) influence the development of modern AI pipelines (e.g., multimodal AI, reinforcement learning agents, or LLM-based systems). What challenges arise when applying classical OOAD to AI-driven architectures?

Answer

Object-Oriented Analysis and Design (OOAD) principles like encapsulation, inheritance, polymorphism, abstraction, and modularity have a profound influence on modern AI pipelines, making them more structured, reusable, and maintainable. However, AI systems introduce unique complexities that sometimes clash with classical OOAD, leading to challenges in application.

Influence of OOAD on AI Pipelines:

1. **Modularity and Decomposition**: OOAD encourages breaking down complex systems into smaller, manageable objects or classes. In AI pipelines, this translates to modular components. For example, in a multimodal AI system (e.g., combining vision and language like CLIP or DALL-E), data ingestion, feature extraction, model training, and inference can be separate classes/modules. A DataLoader class handles input preprocessing, a FeatureExtractor (perhaps inheriting from a base ModelComponent) processes images/text, and an InferenceEngine orchestrates outputs. This modularity allows teams to develop and test components independently, speeding up iteration in fast-evolving AI fields.

2. **Abstraction and Encapsulation**: AI systems often deal with black-box models (e.g., neural networks). OOAD's abstraction hides implementation details behind interfaces. In reinforcement learning (RL) agents like those in AlphaGo or robotic control, an abstract Agent class might define methods like act(state) or learn(reward), encapsulating the underlying policy network or Q-learning algorithm. This allows swapping algorithms (e.g., from DQN to PPO) without affecting the rest of the system. In LLM-based systems like chatbots, a LanguageModel interface encapsulates tokenization, embedding, and generation, making it easier to integrate models from different providers (e.g., GPT vs. Llama).

3. **Inheritance and Polymorphism**: These enable code reuse and flexibility. In AI, a base NeuralNetwork class can be inherited by specialized classes like CNN for vision or Transformer for NLP in multimodal systems. Polymorphism allows a pipeline to treat different models uniformly—e.g., a Trainer class can call train() on any subclass, adapting to RL agents where training involves environment interactions versus supervised learning in LLMs.

4. **Interaction and Collaboration (e.g., via CRC or UML)**: OOAD tools like class diagrams and sequence diagrams model interactions. In AI pipelines, this helps design how an RL agent's Environment class collaborates with the Agent for simulations, or how an LLM pipeline's PromptEngineer interacts with the model for chain-of-thought reasoning.

Overall, OOAD fosters scalable architectures by promoting loose coupling and high cohesion, crucial for AI where models scale to billions of parameters and pipelines handle massive data.

Challenges in Applying Classical OOAD to AI Architectures:

1. **Non-Determinism and Statefulness**: Classical OOAD assumes deterministic behavior, but AI models (e.g., stochastic gradients in training or probabilistic outputs in LLMs) are inherently non-deterministic. Encapsulating this in objects can lead to unpredictable interactions, making debugging hard. For instance, an RL agent's state might depend on random exploration, violating pure encapsulation.

2. **Data-Centric vs. Object-Centric**: OOAD focuses on objects/behavior, but AI is data-driven. In multimodal systems, data flows (e.g., tensors) dominate over objects, leading to challenges in modeling. Traditional classes might not handle dynamic data shapes or distributed computing (e.g., in TensorFlow/PyTorch pipelines), requiring hybrid approaches like functional programming integrations.
3. **Performance Overhead**: Inheritance hierarchies or polymorphic calls introduce overhead, unacceptable in real-time AI (e.g., autonomous agents). Deep inheritance can lead to the "fragile base class" problem when updating core AI components.
4. **Evolving Requirements**: AI development is iterative and experimental; classical OOAD's upfront design (e.g., via UML) can be rigid. Challenges arise in adapting to new architectures like diffusion models or federated learning, where classical decomposition doesn't fit emergent patterns.
5. **Integration with Non-OO Paradigms**: Modern AI often mixes OO with functional (e.g., PyTorch's tensor operations) or procedural code, leading to impedance mismatch. For LLMs, prompt engineering blurs object boundaries, as "behavior" emerges from data rather than code.

In conclusion, while OOAD enhances AI pipelines by providing structure, its classical form must evolve—perhaps via design patterns like Factory for model instantiation or Decorator for layering AI functionalities—to address AI's probabilistic, data-heavy nature. Hybrid methodologies (e.g., combining OOAD with MLOps) are increasingly necessary.

12. Imagine you must teach OOAD concepts using only a diagram—with no text, no labels, no descriptions. Describe the type of diagram you would design, what visual metaphors you would use, and justify how the diagram would still communicate key OOAD concepts (objects, classes, interaction, responsibility).

Answer

To teach OOAD concepts purely visually, I would design a **pictorial analogy diagram**—specifically, a metaphorical scene resembling a "city ecosystem" or "orchestra performance," rendered as a single, integrated illustration (like a comic panel or infographic) without any words. This type of diagram uses symbolic visuals to represent abstract ideas, drawing from real-world analogies to evoke intuition.

Visual Metaphors I Would Use:

1. **Classes as Blueprints/Templates**: Depict a factory or architect's table with a blueprint (a rolled-up scroll or schematic drawing) that gets "stamped" to produce multiple identical houses or robots. The blueprint represents the class (defining structure), while the instantiated houses/robots are objects.

2. **Objects as Instances**: Show multiple copies of the houses/robots in a neighborhood, each slightly customized (e.g., different colors or add-ons) but sharing the core design from the blueprint. This illustrates instantiation and attributes (e.g., color as a variable property).
3. **Responsibilities as Actions/Behaviors**: Each object (house/robot) performs tasks—e.g., a robot watering plants (method: waterPlants()), or a house opening doors (method: openDoor()). Use action lines or motion blur to show these behaviors, emphasizing what the object "does."
4. **Interactions as Connections/Collaborations**: Illustrate objects linking via chains, gears, or handshakes—e.g., one robot passing a tool to another, or houses connected by roads where "messages" (arrows with packages) travel between them. This metaphors method calls and associations.
5. **Overall Composition**: Arrange the scene hierarchically—a central blueprint spawning objects, which then form a dynamic network of interactions, perhaps building a larger structure (like a city) to show system emergence from components.

Justification for Communication Without Text:

- **Intuitive Analogies**: Humans naturally understand blueprints as templates and copies as instances, communicating classes vs. objects without labels. Customization of copies conveys attributes and state.
- **Visual Dynamics for Behaviors**: Actions like "watering" or "passing tools" inherently show responsibilities (what an object knows/does) and interactions (how they collaborate), aligning with CRC concepts. Motion cues (e.g., arrows) imply sequence and dependency.
- **Holistic Storytelling**: The diagram tells a "story" of creation (class to object), individual roles (responsibilities), and teamwork (interactions), mirroring OOAD's core: systems as collaborating objects. Viewers infer key ideas through pattern recognition, much like how UML diagrams work but more accessible via metaphors.
- **Universality**: No text avoids language barriers; visuals like factories or orchestras are culturally broad, allowing learners to map OOAD concepts subconsciously (e.g., "Ah, the blueprint is like a class!").

This approach leverages visual literacy to make OOAD memorable and self-explanatory, proving that good design can transcend words.

13. Conceptual design and technical design often conflict in real-world system development. Identify and explain two deep differences between them using the RU CSE Result Management System as an example.

FarmController + controlSystem(controlFlag: int)public void controlSystem(int controlFlag) {switch (controlFlag) { case 0:return this.startIrrigation(); case 1:return this.startFertilizerDispense(); default:throw new UnknownControlFlagException();}}

In your explanation, show how a misunderstanding between conceptual and technical design could cause system failure.

Answer

Conceptual design focuses on high-level, user-centric models (what the system should do from a domain perspective), while technical design deals with low-level implementation details (how it's built using technology). Using the RU CSE Result Management System (a system for managing student results, grades, and reports in Rajshahi University's CSE department), here are two deep differences:

1. **Abstraction Level and Focus**: Conceptual design is abstract and domain-oriented, emphasizing entities, relationships, and workflows without tech specifics. For example, it might model "Student submits assignment → Faculty evaluates → System computes grade → Report generated," using UML use cases or class diagrams with classes like Student, Assignment, Grade. Technical design is concrete and tech-oriented, specifying databases, algorithms, and code structures. E.g., using SQL for storing grades in a grades_table, Java classes with methods like computeGPA(), and APIs for integration. **Misunderstanding Leading to Failure**: If developers misinterpret the conceptual "grade computation" as a simple average (ignoring technical needs like handling incomplete submissions or weighted rubrics), the system might fail during peak load (e.g., end-of-semester), causing inaccurate reports or crashes due to unoptimized queries. This could lead to administrative chaos, like delayed result publications.

2. **Flexibility vs. Constraints**: Conceptual design is flexible, prioritizing ideal user needs and assuming infinite resources—e.g., real-time result updates for all users, with infinite history access. Technical design imposes real-world constraints like hardware limits, security protocols, or performance trade-offs—e.g., using cached queries for speed but limiting history to 5 years due to storage costs. **Misunderstanding Leading to Failure**: If the conceptual vision of "secure, anytime access for all stakeholders" isn't tempered by technical realities (e.g., implementing OAuth but overlooking database scalability), the system might suffer breaches or slowdowns during high traffic (e.g., result declaration day). This could expose sensitive data or cause downtime, eroding trust and potentially leading to legal issues for the university.

(Note: The provided FarmController code seems unrelated to RU CSE; it might be a remnant from another example, like the Agricultural System. I've focused on the RU CSE context as specified.)

14. Requirements often involve compromises. In the context of the RU CSE Result Management System, discuss at least three situations where requirement compromise becomes unavoidable. For each situation, define:

a. What the user expects

b. What the system can realistically deliver

c. The consequences of choosing one requirement over another

Answer

In developing the RU CSE Result Management System, compromises are inevitable due to budget, time, technical limits, and conflicting stakeholder needs (e.g., students, faculty, admins). Here are three situations:

1. **Real-Time Result Updates vs. System Performance**: a. **User Expects**: Instant, real-time notifications and updates for grades/results as soon as faculty submits them, accessible via mobile app for all users simultaneously. b. **System Delivers**: Batched updates every 15-30 minutes with push notifications, due to server load and database synchronization constraints. c. **Consequences**: Prioritizing real-time could overload servers, causing crashes during peaks (e.g., exam season), affecting reliability. Choosing batched delivery maintains stability but frustrates users expecting immediacy, potentially reducing satisfaction and adoption.

2. **Data Security vs. Ease of Access**: a. **User Expects**: Easy, password-less access (e.g., via university email) with full historical data views for alumni and current users. b. **System Delivers**: Multi-factor authentication (MFA) and role-based access, limiting history to 5 years to comply with data protection laws and reduce storage costs. c. **Consequences**: Emphasizing security prevents breaches but increases login friction, possibly deterring casual users like students checking quick updates. Favoring ease might expose sensitive grades, leading to privacy violations, legal penalties, and loss of trust.

3. **Comprehensive Reporting vs. Development Timeline**: a. **User Expects**: Advanced analytics like predictive GPA trends, customizable reports, and integration with external tools (e.g., LinkedIn for alumni tracking). b. **System Delivers**: Basic reports (e.g., semester-wise PDFs) with core features, postponing advanced analytics to future phases due to time/budget limits. c. **Consequences**: Pushing for comprehensiveness delays launch, missing academic cycles and causing manual workarounds. Opting for basics ensures timely delivery but limits utility, frustrating admins needing insights and requiring costly add-ons later.

These compromises highlight the need for prioritization, often via techniques like MoSCoW (Must-have, Should-have, etc.), to balance ideals with feasibility.

15. Decomposition enables manageable object-oriented systems. Explain why decomposition is essential by imagining a version of the RU CSE Result Management System designed without decomposition. Discuss the problems such a system would face in the areas of scalability, debugging, security, and team collaboration.

Answer

Decomposition in OOAD breaks a system into smaller, independent components (e.g., classes, modules) for better management. Without it, the RU CSE Result Management System would be a monolithic "god class" (e.g., a single ResultSystem handling everything from user login to grade calculation and reporting). This "spaghetti" design would be essential to avoid because it leads to chaos in complex systems.

Problems in a Non-Decomposed System:

1. **Scalability**: The monolithic class would struggle with growth. Adding features (e.g., online proctoring) requires modifying the entire codebase, risking regressions. During high load (e.g., 1000+ students accessing results), the single point of processing could bottleneck, causing slow responses or crashes. Decomposed systems scale by distributing loads (e.g., separate DatabaseHandler for queries, NotificationService for alerts).
2. **Debugging**: Errors would be hard to isolate. A bug in grade calculation might affect login or reporting, as all logic is intertwined. Tracing issues through thousands of lines in one class is time-consuming, increasing downtime. Decomposition allows unit testing individual components (e.g., test GradeCalculator alone), pinpointing faults quickly.
3. **Security**: Vulnerabilities would propagate easily. A flaw in input validation (e.g., SQL injection in user queries) could compromise the whole system, as there's no isolation. Without separate modules, enforcing access controls (e.g., role-based) is messy, risking data leaks. Decomposed designs encapsulate sensitive operations (e.g., AuthModule handles security independently).
4. **Team Collaboration**: Multiple developers working on the same massive class leads to merge conflicts, version control nightmares, and knowledge silos (one person knows the "whole" system). Parallel development is impossible, slowing progress. Decomposition enables teams to own modules (e.g., one on UserInterface, another on DataPersistence), fostering collaboration via clear interfaces.

Question 16

System design and module interaction determine how components cooperate. Choose any modern system you personally use (e.g., Android 2025 OS, ChatGPT mobile app, online banking, or an IoT home device). Explain:

a. How the system is decomposed into modules

b. How these modules interact

c. What would break if module interaction is poorly designed

Answer

I choose the **ChatGPT mobile app (iOS/Android version as of 2025)** as the modern system I use daily.

a. How the system is decomposed into modules

The ChatGPT app is decomposed into loosely coupled, high-cohesion modules typical of modern mobile applications:

- **UI/UX Module**: Handles screens, chat bubbles, input field, settings, theme switching, and animations (built with SwiftUI on iOS or Jetpack Compose on Android).
- **Networking Module**: Manages API calls to OpenAI servers, streaming responses, retry logic, and authentication tokens.
- **Local Storage Module**: Uses Core Data (iOS) or Room (Android) for saving chat history, user preferences, and cached responses.
- **Voice Input/Output Module**: Integrates speech-to-text (Whisper) and text-to-speech for voice mode.
- **Image Processing Module**: Handles image upload, preview, analysis (GPT-4o vision), and camera/gallery integration.
- **Authentication & Subscription Module**: Manages sign-in (Apple/Google/OpenAI), Plus/Team subscription checks, and rate limits.
- **Background Processing Module**: Handles notifications, ongoing streaming when app is in background, and model switching (GPT-4o, o1, etc.).
- **Error Handling & Analytics Module**: Centralised logging, crash reporting, and user behaviour analytics.

b. How these modules interact

Interaction follows clean architectural patterns (mostly MVVM or Clean Architecture):

- Modules communicate primarily through **interfaces**, **events**, or **data flows** rather than direct dependencies.
- The UI module observes a ViewModel that coordinates with use cases in the domain layer.
- Networking module exposes a repository interface that the ViewModel uses to fetch responses.

- When a user sends a message, UI → ViewModel → Repository → Networking → streams back tokens → UI updates in real-time.
- Voice module interacts with UI via delegates and with networking for Whisper transcription.
- Local storage is accessed via repository pattern to keep UI independent of database details.

c. What would break if module interaction is poorly designed

Poor interaction (e.g., tight coupling, direct calls, god objects) would cause:

- **Fragile updates**: Changing the API endpoint would require modifying UI code directly → high risk of crashes during server updates.
- **Slow performance**: If UI directly handles networking without abstraction, background threads aren't managed properly → frozen interface or ANR (Application Not Responding) errors.
- **Data loss**: Without proper separation, background module might save chats incorrectly if networking fails → lost conversation history.
- **Security issues**: Authentication details leaking into UI or analytics modules → token exposure.
- **Hard to add features**: Adding voice mode or image analysis would require touching many unrelated files → bugs in existing chat functionality.

Real-world impact: users experience crashes, lost chats, delayed responses, or inconsistent behaviour across updates—leading to poor reviews and churn.

Question 17

Explain the concept of Separation of Concerns (SoC) using a feature of your Android phone (2025 version). Pick one specific feature (e.g., biometric authentication, notification system, camera pipeline, AI voice assistant) and explain how SoC prevents complexity and failures.

Answer

I will use the **Camera Pipeline** feature in Android 2025 (e.g., on Pixel 10 or Samsung Galaxy S25).

Concept of Separation of Concerns (SoC): SoC is a design principle that divides a system into distinct sections, each addressing a separate concern, with minimal overlap. This reduces complexity, improves maintainability, and limits the impact of changes.

Application in Android Camera Pipeline:

The camera system is separated into several independent concerns:

1. **Hardware Abstraction Layer (HAL)**: Deals only with physical camera sensors, lens control, and raw data capture.
2. **Image Processing Pipeline (ISP)**: Handles concerns like noise reduction, HDR merging, colour correction, and AI enhancements (Night Sight, portrait mode).
3. **Camera Framework (CameraX/Camera2 API)**: Provides a clean interface for apps, managing session lifecycle, preview, and capture requests.
4. **App-Level Concern**: The Camera app or third-party apps handle UI, user settings (zoom, flash), storage, and sharing.
5. **AI/ML Module**: Separate concern for features like scene detection, object recognition, or real-time video effects (e.g., Magic Eraser).

How SoC prevents complexity and failures:

- **Complexity reduction**: A developer fixing a low-light noise issue works only in the ISP module without touching UI or storage code.
- **Independent evolution**: Google can improve Night Sight AI without breaking basic photo capture or third-party app compatibility.
- **Failure isolation**: If the AI scene detection module crashes, basic photo and video recording still work. In a non-SoC design (everything mixed), one bug could make the entire camera unusable.
- **Testing & security**: Each concern can be unit-tested separately; sensitive hardware access is isolated in HAL, reducing attack surface.

Without SoC, a single change (e.g., adding a new filter) could unintentionally break autofocus or video recording, leading to widespread user complaints and emergency patches.

Question 18

Conceptual integrity ensures that a system feels consistent. Using the RU CSE Result Management System, describe:

a. A scenario where lack of conceptual integrity creates confusion for users or developers

b. How enforcing conceptual integrity would avoid these issues

c. A design rule you would apply to maintain conceptual integrity

Answer

a. Scenario where lack of conceptual integrity creates confusion

Faculty members enter marks using a web portal, while exam controllers use a separate desktop application with different workflows and terminology. Students view results via a mobile app that uses yet another naming scheme (e.g., "Grade Point" in one place, "CGPA" inconsistently, "Marks" vs "Score"). Some screens show letter grades (A+, A), others show numerical GPA, and error messages vary wildly ("Invalid data" vs "Bad input detected" vs "Entry failed").

Result: Faculty get confused about which tool to use for what, students misinterpret their performance, and developers struggle to understand which component owns "grade calculation" logic.

b. How enforcing conceptual integrity would avoid these issues

A single unifying concept—"Result" as a core entity with consistent attributes (course code, student ID, marks, grade point, letter grade)—would be adopted across all interfaces. All user-facing terms, workflows (e.g., marks entry → moderation → publication), and visual styles would follow the same mental model. This ensures faculty, students, and admins experience the system as one coherent whole rather than fragmented tools.

c. Design rule to maintain conceptual integrity

"**One Concept, One Representation**": Every major domain concept (e.g., Course, SemesterResult, GradeScale) should have exactly one primary class/module and one consistent terminology/presentation across the entire system. Any deviation must be justified and approved in design reviews. This rule, inspired by Fred Brooks' ideas in *The Mythical Man-Month*, prevents proliferation of synonyms and duplicate logic.

Question 19

Explain the importance of design thinking and object-oriented thinking using a short original example. Your example should demonstrate how a poorly designed system behaves differently from a well-designed one built using OO principles.

Answer

Example: A small university event ticketing system.

Poorly Designed (Procedural, no OO thinking): Everything is in one large script/file with functions like checkSeatAvailability(eventID), bookTicket(eventID, userID, seatNumber), sendConfirmationEmail(userID). All data (events, seats, users) is stored in global arrays or a single large dictionary. Adding a new feature like "group booking discount" requires modifying multiple functions, duplicating logic, and hunting through the file.

Behaviour:

- Adding student discount → edit three different functions → accidentally break email sending.

- Two developers working simultaneously → frequent merge conflicts.
- System becomes slower and buggier as events increase.

Well-Designed (Using OO Principles): Classes: Event (contains list of Seat objects), Ticket, User (Student or General), BookingService.

- Event encapsulates seat availability.
- User subclasses: StudentUser overrides calculatePrice() for discount.
- BookingService coordinates: creates Ticket, updates Event, notifies user.

Behaviour:

- Adding group discount → create GroupBooking subclass or decorator → no changes to existing classes.
- Bugs are isolated (e.g., email issue stays in notification class).
- New developers understand responsibilities quickly via class names and interfaces.

Importance: Object-oriented thinking forces us to model real-world entities and responsibilities clearly, leading to systems that are easier to extend, debug, and maintain. Design thinking complements this by ensuring we first understand user needs (e.g., students want discounts) before jumping to code—resulting in systems that are both technically sound and genuinely useful.

Question 20

Using the RU CSE Result Management System as the scenario, explain the following with specific examples:

a. Abstraction

b. Encapsulation

c. Decomposition

d. Generalization

Each explanation must refer to real departments, courses, reports, or workflows in a result management context.

Answer

a. Abstraction Abstraction hides complex implementation details and exposes only essential features.

Example: The process of generating a semester transcript report. Users (students or admins) call a simple method Transcript.generateFor(studentID, semester) without knowing the internal steps:

fetching marks from multiple courses (CSE301, CSE302, etc.), applying grade scale of RU CSE department, calculating SGPA, checking pre-requisite clearances, and formatting as PDF. The complex database queries and business rules are abstracted behind the Transcript service interface.

b. Encapsulation Encapsulation bundles data and methods together while restricting direct access to internals.

Example: The CourseMarks class for a course like CSE405 (Software Engineering). It holds private attributes: caMarks, midtermMarks, finalMarks, totalMarks. Public methods like setCAMarks(value) include validation (0–40 range), and getGradePoint() computes internally using the RU CSE grade policy. External classes (e.g., faculty portal) cannot directly modify totalMarks—they must use setters that enforce rules—preventing invalid data entry.

c. Decomposition Decomposition breaks the system into smaller, manageable modules with single responsibilities.

Example: Instead of one huge system, the result workflow is decomposed:

- MarksEntryModule (faculty enters continuous assessment),
- ModerationModule (exam committee reviews),
- GradeProcessingModule (applies scale and computes SGPA),
- TabulationSheetGenerator (produces departmental reports),
- PublicationModule (releases results to students).

This allows parallel development and isolates changes (e.g., changing moderation rules affects only one module).

d. Generalization Generalization (inheritance/specialization) identifies common features in subclasses and places them in a parent class.

Example: Both undergraduate (BSc) and postgraduate (MSc) students in RU CSE have results, but with different credit requirements and thesis components. A parent class StudentResult contains common attributes/behaviour: studentID, semester, courseList, calculateSGPA(). Subclasses UndergradResult and PostgradResult inherit these and add specifics—UndergradResult enforces 160 total credits, while PostgradResult includes thesisMarks and overrides calculateCGPA() to weight thesis differently. This avoids code duplication across programs.